

Creating a React App: Hinglish Explanation

Is page ka main message simple hai:

Naya production-ready React app banana ho, toh React team generally framework se start karne ki recommendation karti hai.

Framework routing, data fetching, rendering aur deployment jaise common decisions ke liye ready-made solutions provide karta hai. Lekin fundamentals seekhne, custom requirements handle karne, ya apna architecture banane ke liye React app scratch se bhi start kar sakte ho.

1. React app start karne ke do approaches

Approach A: Framework use karo

Framework tab useful hai jab tum:

- Production-ready website ya app bana rahe ho
- Routing aur data fetching manually configure nahi karna chahte
- App ko future mein deploy aur scale karna chahte ho
- Client aur server features ko ek saath use karna chahte ho

Page par recommended options hain:

1. **Next.js App Router**
2. **React Router v7**
3. **Expo**, native mobile apps ke liye

Approach B: Scratch se React app banao

Scratch se start karna useful ho sakta hai agar:

- Existing frameworks tumhari requirements properly handle nahi karte
- Tum apna framework ya architecture banana chahte ho
- Tum React setup ke fundamentals samajhna chahte ho
- Tum routing, data fetching aur tooling ke decisions khud lena chahte ho

Is approach mein flexibility zyada hoti hai, lekin configuration aur technical decisions bhi zyada lene padte hain.

2. “Full-stack framework” ka kya matlab hai?

Full-stack React framework tumhe frontend ke saath server-related features use karne ki facility bhi deta hai.

Lekin page ek important clarification deta hai:

Full-stack framework use karne ka matlab yeh nahi hai ki tumhe compulsory server chalana padega.

Page par diye gaye frameworks ye rendering strategies support karte hain:

CSR: Client-Side Rendering

Browser JavaScript run karke page ka content render karta hai.

Ye interactive applications aur browser-focused experiences mein use ho sakta hai.

SPA: Single-Page Application

Application generally ek page load karti hai. Uske baad navigation ke time complete page reload karne ki jagah JavaScript UI ko update karta hai.

SSG: Static-Site Generation

Pages build time par generate ho jaate hain. Generated static files ko CDN ya static hosting par deploy kiya ja sakta hai.

In approaches ke liye dedicated runtime server zaroori nahi hota.

Per-route server-side rendering

Frameworks zarurat ke according specific routes par server-side rendering add karne dete hain.

For example:

- /about static ho sakta hai
- /dashboard server-rendered ho sakta hai
- /settings client-side interactive route ho sakta hai

Iska benefit yeh hai ki tum client-only app se start kar sakte ho. Agar baad mein server features ki zarurat aaye, toh selected routes par unhe enable kar sakte ho, bina poori application rewrite kiye.

3. Next.js App Router

Next.js App Router ek React framework hai jo React ki architecture ka use karke full-stack applications banane deta hai.

Naya Next.js project create karne ke liye:

```
1 npx create-next-app@latest
```

Page ke according:

- Next.js ko Vercel maintain karta hai
- Ise Node.js support karne wale hosting provider par deploy kiya ja sakta hai
- Docker container ke through deploy kiya ja sakta hai
- Apne server par host kiya ja sakta hai
- Static export bhi supported hai
- Static export ko server ki zarurat nahi hoti

Next.js App Router ki special importance

Page batata hai ki Next.js App Router official React Server Components specification ko fully implement karta hai.

Isse ek hi React component tree mein tum mix kar sakte ho:

- Build-time components
 - Server-only components
 - Browser mein chalne wale interactive components
-

4. Server Components kaise kaam karte hain?

Page par yeh example diya gaya hai:

```
1 async function Talks({ confId }) {  
2   const talks = await db.Talks.findAll({ confId });  
3  
4   const videos = talks.map(talk => talk.video);  
5  
6   return <SearchableVideoList videos={videos} />;  
7 }
```

Is example ko step-by-step samjho.

Step 1: Component server par run hota hai

```
1 async function Talks({ confId })
```

Yeh ek asynchronous React component hai jo server par ya build ke time run hota hai.

Browser mein directly execute nahi hota.

Step 2: Database ko directly access karta hai

```
1 const talks = await db.Talks.findAll({ confId });
```

Kyunki component server environment mein hai, isliye yeh data layer ya database se directly communicate kar sakta hai.

Page specifically batata hai ki is situation mein separate API endpoint required nahi hai.

Step 3: Server par rendering logic execute hoti hai

```
1 const videos = talks.map(talk => talk.video);
```

Data transformation server par ho rahi hai. Yeh logic browser ke JavaScript bundle ko bada nahi karta.

Step 4: Data interactive component ko diya jata hai

```
1 return <SearchableVideoList videos={videos} />;
```

Server Component processed data ko browser mein chalne wale interactive component tak pass kar sakta hai.

Matlab:

- Database access server par
 - Data processing server par
 - Search aur interaction browser mein
-

5. Suspense aur loading UI

Next.js App Router data fetching ko React Suspense ke saath integrate karta hai.

Page par example hai:

```
1 <Suspense fallback={<TalksLoading />}>
2   <Talks confId={conf.id} />
3 </Suspense>
```

Iska matlab:

- <Talks /> data load karta hai
- Jab tak data ready nahi hota, <TalksLoading /> show hota hai
- Data ready hone ke baad actual Talks UI display hoti hai

fallback ke andar tum loading indicator ya skeleton placeholder rakh sakte ho.

Suspense ka benefit yeh hai ki alag-alag sections ke loading states React component tree ke andar define kiye ja sakte hain.

Important distinction:

Server Components aur Suspense React ke features hain, sirf Next.js ke features nahi.

Lekin in features ko framework level par properly implement karna complex hota hai. Page ke according, Next.js App Router filhaal inka sabse complete implementation provide karta hai.

6. React Router v7

React Router ko page React ki sabse popular routing library batata hai.

React Router v7 ko Vite ke saath pair karke full-stack React framework project create kiya ja sakta hai.

Project create karne ke liye:

```
1 npx create-react-router@latest
```

Page ke according React Router:

- Standard Web APIs par emphasis karta hai
- Different JavaScript runtimes aur platforms ke liye ready-to-deploy templates provide karta hai
- Shopify ke through maintain kiya jata hai

React Router ka naam sirf routing jaisa lag sakta hai, lekin page specifically v7 ke framework mode ki baat kar raha hai.

7. Expo: Android, iOS aur web apps ke liye

Agar tum React ka use karke native mobile application banana chahte ho, toh page **Expo** recommend karta hai.

Expo ki help se universal apps banayi ja sakti hain:

- Android
- iOS
- Web

Naya Expo project create karne ke liye:

```
1 npx create-expo-app@latest
```

Expo React Native ke liye SDK provide karta hai, jisse native functionality use karna easier ho jata hai.

Page ke according:

- Expo se app banana free hai
 - Apps Google aur Apple app stores mein submit ki ja sakti hain
 - Submission par Expo ki taraf se restrictions nahi hain
 - Optional paid cloud services available hain
-

8. Other upcoming frameworks

Page do aur frameworks mention karta hai jo React ke full-stack vision ki taraf kaam kar rahe hain.

TanStack Start

Page par TanStack Start ko **Beta** bataya gaya hai.

Yeh TanStack Router par powered full-stack React framework hai. Mentioned features hain:

- Full-document SSR
- Streaming
- Server functions
- Bundling
- Nitro aur Vite jaise tools ka use

RedwoodSDK

RedwoodSDK ek full-stack React framework hai.

Page ke according, ismein kaafi packages aur configuration pehle se installed hote hain, jisse full-stack web applications banana easier hota hai.

9. React app scratch se banana

Framework compulsory nahi hai.

Agar tum scratch se React project start karte ho, toh tumhe common decisions khud lene honge, jaise:

- Routing ke liye kya use karna hai

- Data fetching kaise karni hai
- Project build kaise hoga
- Common application patterns kaise organize honge

Isi wajah se page scratch setup ko apna framework banane jaisa compare karta hai.

Scratch se project banane ke liye page ye build tools mention karta hai:

- Vite
- Parcel
- RSbuild

Framework mein common problems ke built-in solutions milte hain. Scratch setup mein flexibility milti hai, lekin un solutions ko choose aur configure karna developer ki responsibility hoti hai.

Practical decision: tumhe kya choose karna chahiye?

Page ki information ko practical decision mein convert karein toh:

Normal React website ya production full-stack app

Next.js App Router consider karo.

Especially jab tum server rendering, Server Components, Suspense aur full-stack capabilities use karna chahte ho.

Web APIs ke close rehna hai aur React Router framework use karna hai

React Router v7 consider karo.

Android/iOS native app banana hai

Expo choose karo.

React basics aur configuration seekhni hai

Scratch setup se start kar sakte ho, using Vite, Parcel ya RSbuild.

Maximum control chahiye

Scratch setup suitable ho sakta hai, lekin routing aur data fetching jaise decisions khud handle karne padenge.

Sabse important takeaway

Purane tutorials mein tumhe create-react-app mil sakta hai, lekin is supplied page mein uski recommendation nahi di gayi.

Page ka suggested direction hai:

- 1 Production web app
- 2 → Next.js App Router ya React Router
- 3
- 4 Native Android/iOS app
- 5 → Expo
- 6
- 7 React fundamentals ya custom setup
- 8 → Scratch setup with Vite, Parcel, or RSbuild

Framework ka use karne ka matlab compulsory server nahi hai. Tum client-only ya static app se start karke later individual routes par server features add kar sakte ho.

Page mein diye gaye external links

Mozilla/MDN links ko omit karke page mein ye external websites linked hain:

- [Next.js documentation](#)
- [Vercel](#)
- <https://nextjs.org/docs/app/building-your-application/deploying>
- [Next.js static exports](#)
- [React Router installation](#)
- <https://github.com/remix-run/react-router-templates>
- [Shopify](#)
- [Expo](#)
- [React Native](#)
- [Expo tutorial](#)
- <https://expo.dev/about>
- [TanStack Start](#)
- <https://rwsdk.com/>
- <https://github.com/reactjs/rfcs/blob/main/text/0188-server-components.md>
- [Vite](#)
- <https://parceljs.org/>
- [RSbuild](#)
- <https://github.com/reactjs/react.dev/issues/new?assignees=&labels=type%3A+framework&projects=&template=3-framework.yml&title=%5BFramework%5D%3A+>